MCT344
Industrial Robotics

Faculty of Engineering - Ain Shams University
ICHEP
International Credit Hours Engineering Programs

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

International Credit Hours Engineering Programs (I.CHEP)

MCT344

Industrial Robotics

First Milestone

Supervised By:

Dr.Shady Ahmed,

Eng.Mina Yousry,

Eng. Dina Zakaria.

Team Members

Name	ID
Philopateer Elkes	21P0240
Yahia Mohamed	21P0041

Table of Contents

Abstract: 4

1.0 Introduction 5

2.0CAD Model Import & URDF Conversion 6

4.0 Configuring ROS2 Controllers 8

5.0 Creating the Launch File 10

6.0 Drive Link 13

List of figures:

Figure 1:CAD Model 6

Figure 2:Joint Names 7

Figure 3:Joint controller.yaml 8

Figure 4:Joint controller.yaml 2 9

Figure 5:Launch file 1 10

Figure 6:Launch file 2 11

Figure 7:Launch file 3 12

Figure 8:Robot in Gazebo 13

Abstract:

This report presents the modeling and simulation of an ABB IRB 1200 robotic arm with a custom-designed two-joint gripper end-effector in the ROS2 and Gazebo environments. The objective of this milestone was to develop a 6-DOF manipulator using the Unified Robot Description Format (URDF), ensuring compatibility with Gazebo for realistic physics-based simulation. The project involved defining the robot's kinematics, visual properties, and dynamics, configuring ROS controllers for joint actuation, and integrating the model into a launch system for seamless execution in Gazebo. The final implementation allows for controlled movements of the robotic arm and gripper using ROS2 commands, demonstrating successful simulation and validation.

1.0 Introduction

Industrial robotic arms are widely used in automation, manufacturing, and precision assembly, requiring accurate simulation for design validation and control testing. In this milestone, we focused on modeling and simulating the ABB IRB 1200 robotic arm, a widely used 6-DOF manipulator, and designing a two-joint gripper end-effector to extend its capabilities.

Using ROS2 and Gazebo, we developed a URDF-based robot model, defining its links, joints, and physical properties. The simulation was enhanced by incorporating joint constraints, dynamic parameters, and ROS controllers, ensuring smooth and realistic operation. A .yaml configuration file was created to define the necessary joint controllers, and a .launch.py script was implemented to deploy the robot in Gazebo efficiently.

This report outlines the design process, simulation setup, controller configuration, and validation steps. Key challenges, such as URDF errors, Gazebo physics tuning, and controller integration, are discussed along with solutions implemented to ensure a functional and responsive robotic system. The final result is a realistic simulation of the ABB IRB 1200 with a working two-joint gripper, capable of controlled movements within a virtual environment.

2.0CAD Model Import & URDF Conversion

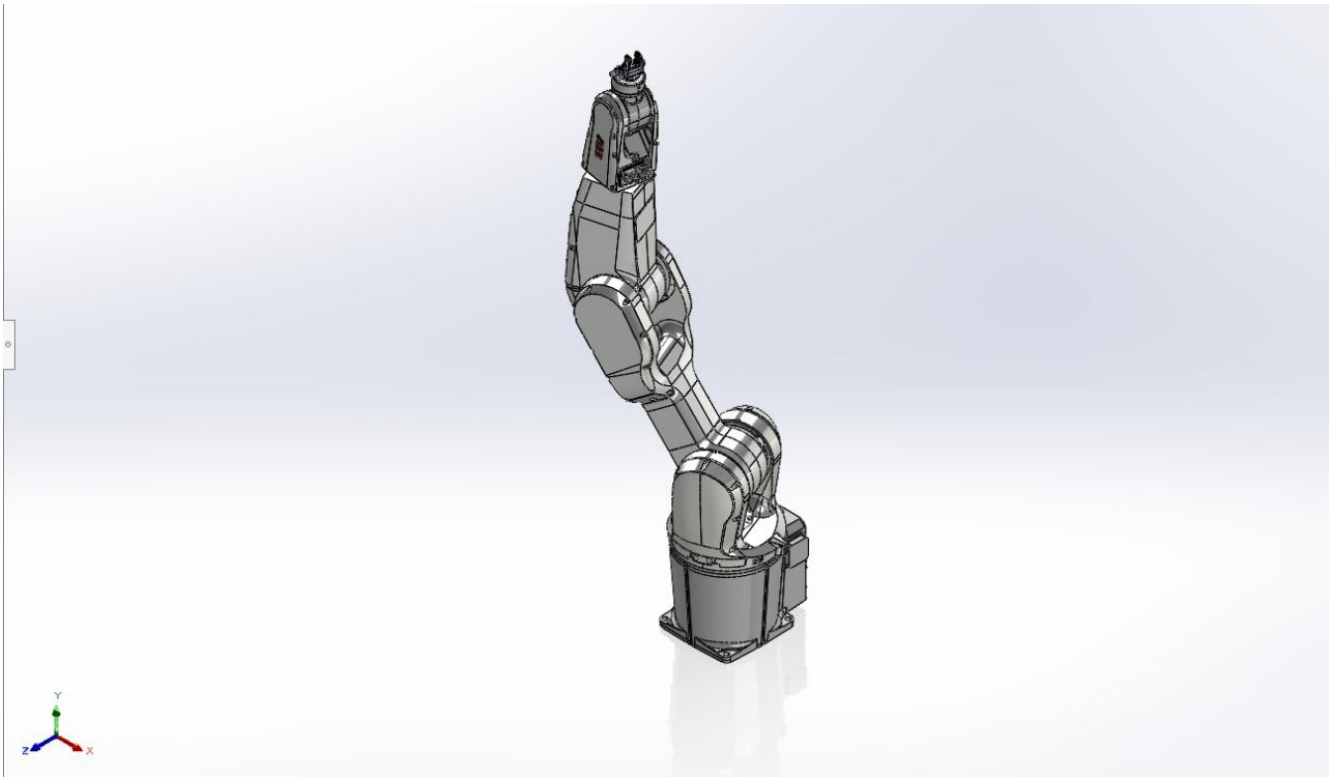

Figure 1:CAD Model

To integrate the robotic arm with ROS2, we exported the SolidWorks assembly as a URDF file using the SolidWorks URDF Exporter Plugin. During the export process, each link and joint was carefully named and assigned realistic mass and inertia values.

```xml
</hardware>
<joint name="joint1_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint2_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint3_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint4_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint5_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
```

Figure 2:Joint Names

4.0 Configuring ROS2 Controllers

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100 # Hz
    arm_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    hand_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

arm_group_controller:
  ros__parameters:
    joints:
      - joint1_21P0041_21P0240
      - joint2_21P0041_21P0240
      - joint3_21P0041_21P0240
      - joint4_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    action_monitor_rate: 20.0
    allow_partial_joints_goal: false
    open_loop_control: true
    constraints:
      stopped_velocity_tolerance: 0.01
      goal_time: 0.0
      joint1_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint2_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint3_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint4_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
```

Figure 3:Joint controller.yaml

```yaml

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
      - joint7_21P0041_21P0240
      - joint8_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    gains:
      joint5_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }
      joint6_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }

```

Figure 4:Joint controller.yaml 2

5.0 Creating the Launch File

```python
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    IncludeLaunchDescription,
    SetEnvironmentVariable,
)
from launch.substitutions import PathJoinSubstitution
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import xacro
import os


def generate_launch_description():
    pkg_ros_gz_sim = get_package_share_directory("ros_gz_sim")
    pkg_lab_gazebo = get_package_share_directory("abb_robot")

    # Process the Xacro file to generate URDF
    xacro_file = os.path.join(pkg_lab_gazebo, "urdf", "robot_pkg.urdf")
    doc = xacro.parse(open(xacro_file))
    robot_description = doc.toxml()

    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_ros_gz_sim, "launch", "gz_sim.launch.py")
        ),
        launch_arguments={"gz_args": "-r empty.sdf"}.items(),
    )

    # Robot State Publisher node
    robot_state_publisher_node = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description}],
        output="screen",
    )

    # Joint State Publisher node
    joint_state_publisher_node = Node(
        package="joint_state_publisher",
        executable="joint_state_publisher",
        name="joint_state_publisher",
        output="screen",
    )

    # Add the bridge node for mapping the topics between ROS and GZ sim
```

Figure 5:Launch file 1

```python
        package="ros_gz_bridge",
        executable="parameter_bridge",
        arguments=[
            # Clock (IGN -> ROS2)
            "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock",
            # Joint states (IGN -> ROS2)
            "/world/empty/model/abb_robot/joint_state@sensor_msgs/msg/JointState[gz.msgs.JointState",
        ],
        remappings=[
            ("/world/empty/model/abb_robot/joint_state", "joint_states"),
        ],
        output="screen",
    )

    # Spawn Entity node
    spawn_entity = Node(
        package="ros_gz_sim",
        executable="create",
        arguments=["-name", "abb_robot", "-topic", "/robot_description"],
        parameters=[{"use_sim_time": True}],
        output="screen",
    )

    # add the robot controllers for joint trajectory, arm group and hand group controllers
    robot_controller = PathJoinSubstitution( [
        pkg_lab_gazebo,
        "config",
        "joint_controller.yaml", ]
    )

    # pass the controllers to the control node to send commands to the controllers
    control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[robot_controller],
        output="screen",
    )

    # Controller Manager Spawner node to launch the joint state broadcaster
    joint_state_broadcaster_spawner = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
```

Figure 6:Launch file 2

```python
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['arm_group_controller',"--param-file", robot_controller], output='screen'
    )
    # Controller Manager Spawner node to launch the hand group controller
    hand_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['hand_group_controller', "--param-file", robot_controller], output='screen'
    )

    # Get the meshes path in the install directory
    meshes_path = os.path.join(pkg_lab_gazebo, "meshes")
    return LaunchDescription(
        [
            SetEnvironmentVariable("GZ_SIM_RESOURCE_PATH", meshes_path),
            gazebo,
            spawn_entity,
            bridge,
            robot_state_publisher_node,
            joint_state_publisher_node,
            control_node,
            arm_controller_manager_spawner,
            hand_controller_manager_spawner,
            joint_state_broadcaster_spawner,
        ]
    )

```

Figure 7:Launch file 3

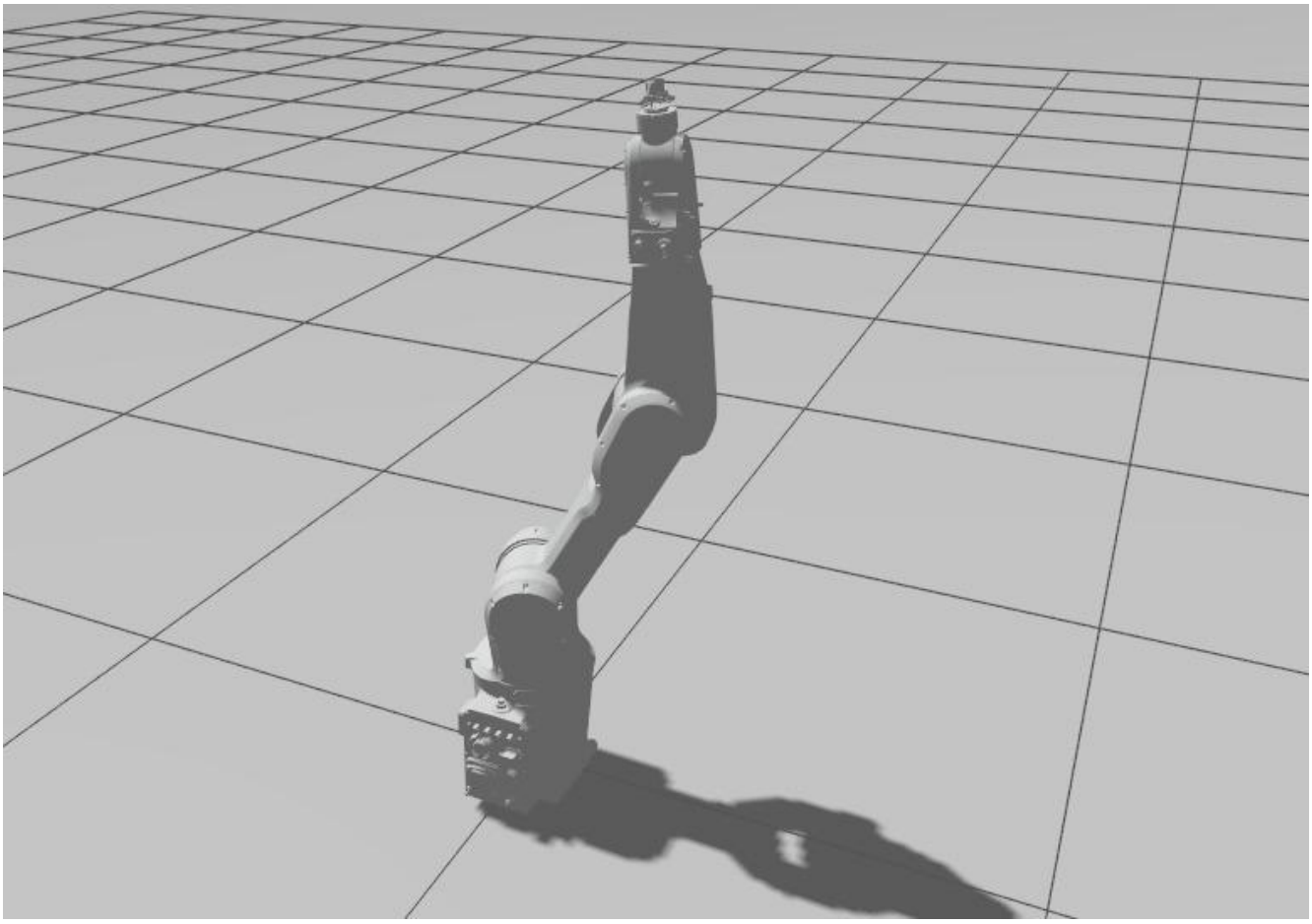

Figure 8:Robot in Gazebo

6.0 Drive Link

https://drive.google.com/drive/folders/1Bjn4gtHMUtRtRiE3VrS5pSFjiBGGSxiS?usp=sharing